

openrcs — user guide

openrcs is a modern control surface for Analog Way **Midra** and **LiveCore** series video processors. A small bridge server holds one connection to the processor and serves a web UI to any number of browsers, so you can drive the device from a laptop, a tablet, or a touch panel — no vendor runtime required.

Running it

Start the bridge server, pointing it at the processor's control port (TCP 10500), and open the web UI:

```
cargo run -p openrcs-server -- --device <processor-ip>:10500 --platform livecore
# then open http://127.0.0.1:8730/
```

Use `--platform midra` for the Midra family. The header shows the device model, platform and a connection indicator; every view updates live as the device — or another operator — changes state.

`--device` is optional. Started without one, the server comes up unconfigured and shows only the **Connection** view: enter the processor's address on the on-screen keypad, or press **Scan** to look for processors on the network, pick the platform and press **Connect**. The choice is written to a config file (`~/.config/openrcs/config.json` by default, or `--config <file>`) and used on the next start, so this only has to be done once. Connection also retargets a running server — useful when one surface covers more than one processor.

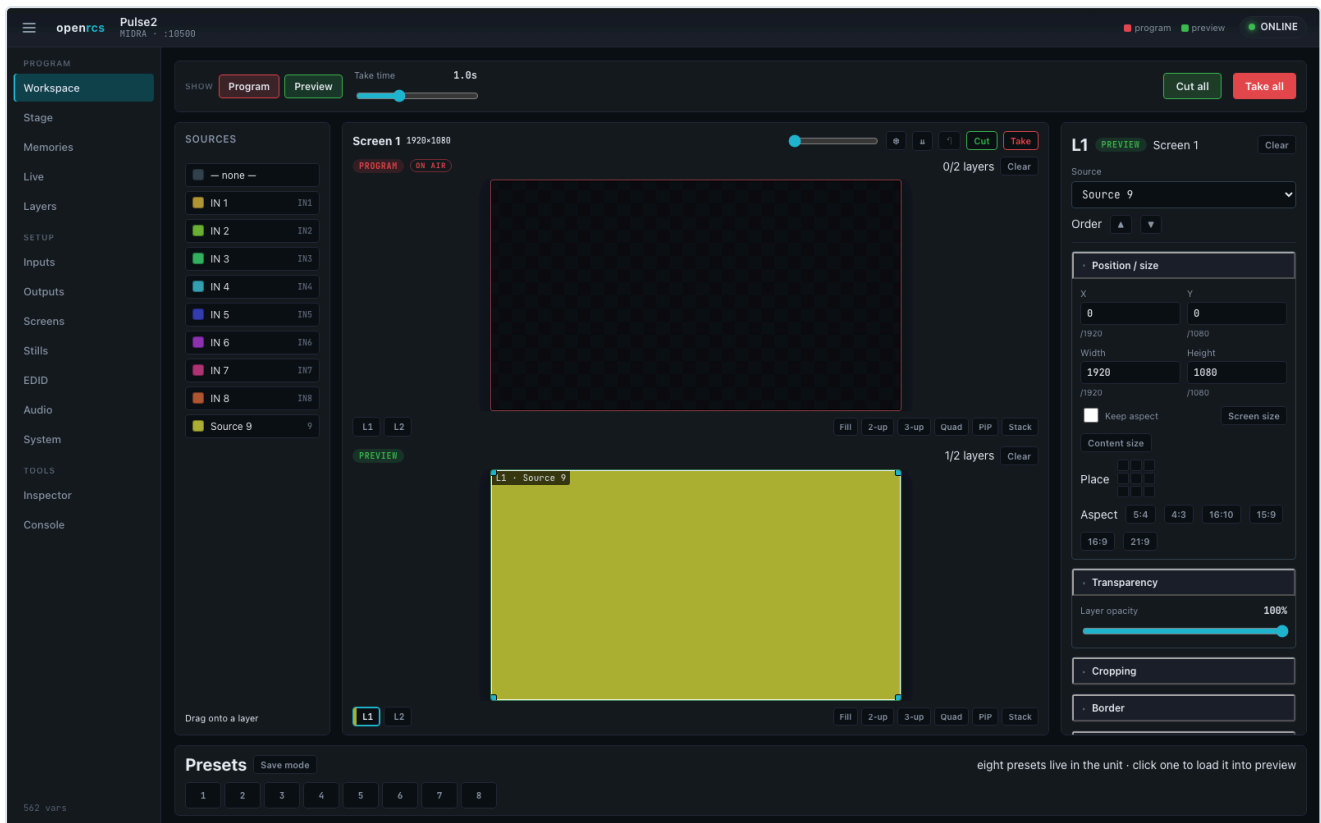
The scan is read-only: it opens a connection, listens for anything the processor volunteers, and never writes to what it finds — so it is safe to run on a live network. Some processors identify themselves that way and some say nothing at all; a result listed as **unidentified** was found and answered, it just did not name its platform, so pick that yourself and connect. A processor that already has a control session open elsewhere may not answer at all.

No command line? The [tray launcher](#) (macOS, Windows, Linux) bundles the server in a menu-bar app: enter the switcher's IP, pick the model, click **Start**, then **Open**. Nothing else to install.

The interface is a single dark theme, chosen deliberately for the blacked-out environments these processors live in. The left nav is grouped into **Program** (the things you touch during a show), **Setup** (configuration), and **Tools**.

It adapts to the device. openrcs reads the variable table the processor advertises and shows only the views that hardware actually supports — so a LiveCore unit exposes the full set below, while a Midra unit shows the subset it implements (and swaps in its own equivalents, e.g. a per-screen

Freeze where LiveCore offers a master fade). The two families model some things differently — memories especially — and the UI follows each one's model rather than forcing a single shape.



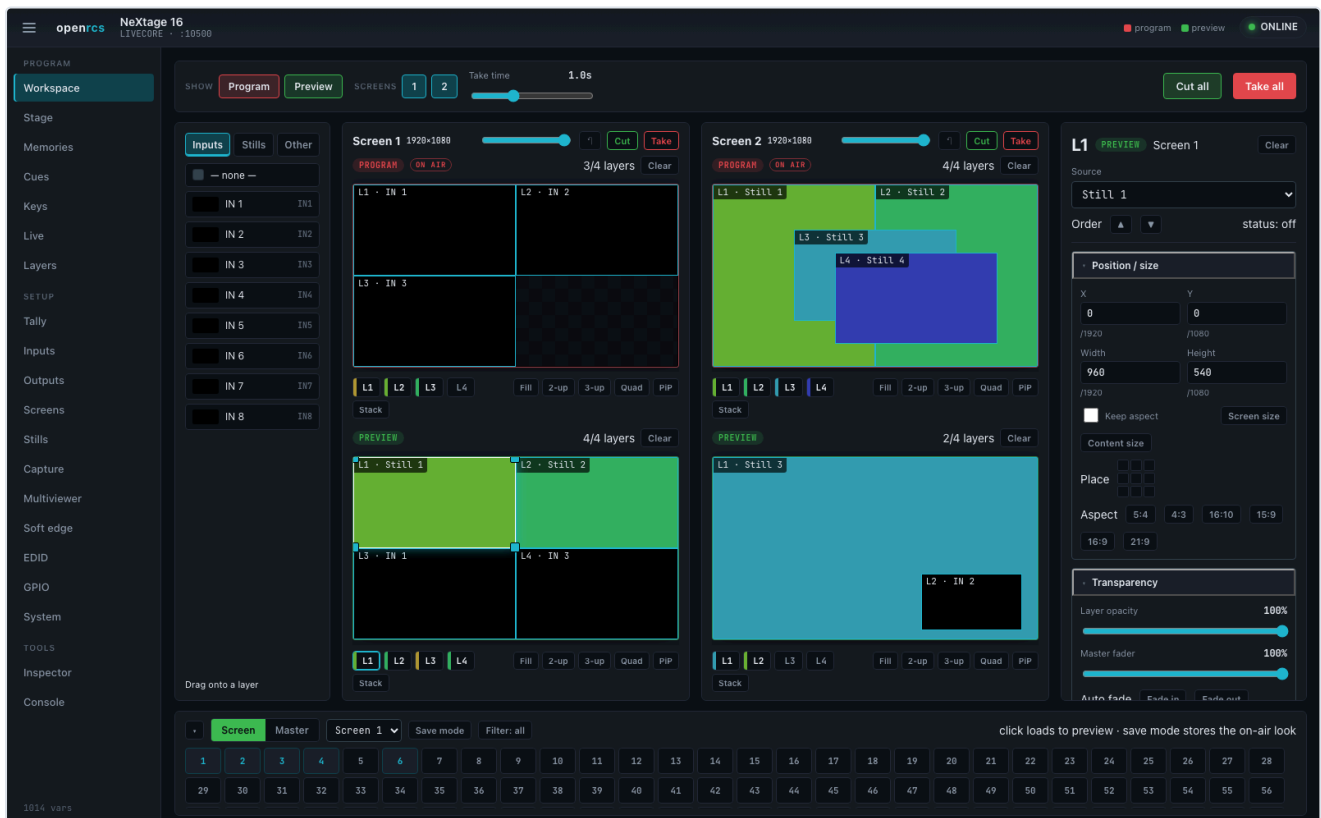
New in this release

- **Layer memories** — a fourth memory bank beside master and screen. Capture one layer's whole property set and apply it to any screen, preset and layer, under the same category filter as a screen memory. No processor stores a single layer, so this bank lives in the browser: it survives a reload, travels in a show file, and every recall is checked by reading the layer back — which is how it catches a Midra refusing a source that has no signal.
- **Memories** gains what only the Workspace panel had: each slot's label as stored on the device, erase, an explicit preview-or-program target, and the device's own twelve-category record filter applied to every recall.
- **Video out** (Midra) — the frame's second output, on its own view. Choose what its plug is for, pick which screen it looks at, and drag an **area of interest** to send a crop of that screen rather than the whole thing. See the note below about what it can and cannot carry.
- **Area of interest** on each LiveCore output — staged, then committed with Apply. Unconfirmed on hardware; the panel shows the device's own readback rather than the numbers you typed, and says so.
- **Working area** — a region of a screen that openrcs composes inside. Layouts divide it, and no drag, resize, typed value or memory recall can put a layer outside it. Nothing is written to the processor to set one up. Use it when only part of a screen is really seen — an LED wall inside a larger canvas, or a feed that has to stay inside a frame.

- **Show mode** — a stripped, big-target front-of-house surface: large CUT ALL / TAKE ALL, a TAKE tile per destination, and a grid of master-memory recall tiles. The one to drive a show from a touchscreen at front-of-house.
- **Wall** — where each screen sits in the output. Every screen is placed in the output-tile grid at its real position; drag to rearrange, then apply.
- **Destinations** — take, cut, T-bar and step-back whole **screen groups** as one destination, with a grouping editor and a TAKE ALL GROUPS.
- **Shows** — capture the device's writable state (the live look, the memory banks, input or output setup) to a portable JSON file and restore it. Restore re-reads the device first and writes only what differs. Also hosts **Confidence**, an instant cache-based undo that can auto-snapshot before every take.
- **Plan** — build a look with no device attached (reads preview your staged values), then push it to the processor on connect. Staged work persists across a reload; "Seed from look" starts from the current on-screen state.
- **Cues** now chain: mark a cue **autofollow** and it runs the next one after a per-cue wait, with a **HOLD** to pause and per-cue notes.

On LiveCore, a **take** now animates the T-bar directly rather than firing the device's own take verbs — on real hardware those leave the group stuck mid-transition, so openrcs sweeps the bar over the transition time instead.

Workspace



One page that mirrors how you actually drive a show, with everything in reach: the **source palette** down the left, **every active screen** in the middle with its program above its preview, and the **layer properties** panel on the right. The page fits the window — the previews grow to fill the space — and the memory strip along the bottom folds away when you want more room. Collapse the main menu with the button top-left (it stays collapsed) for more space still.

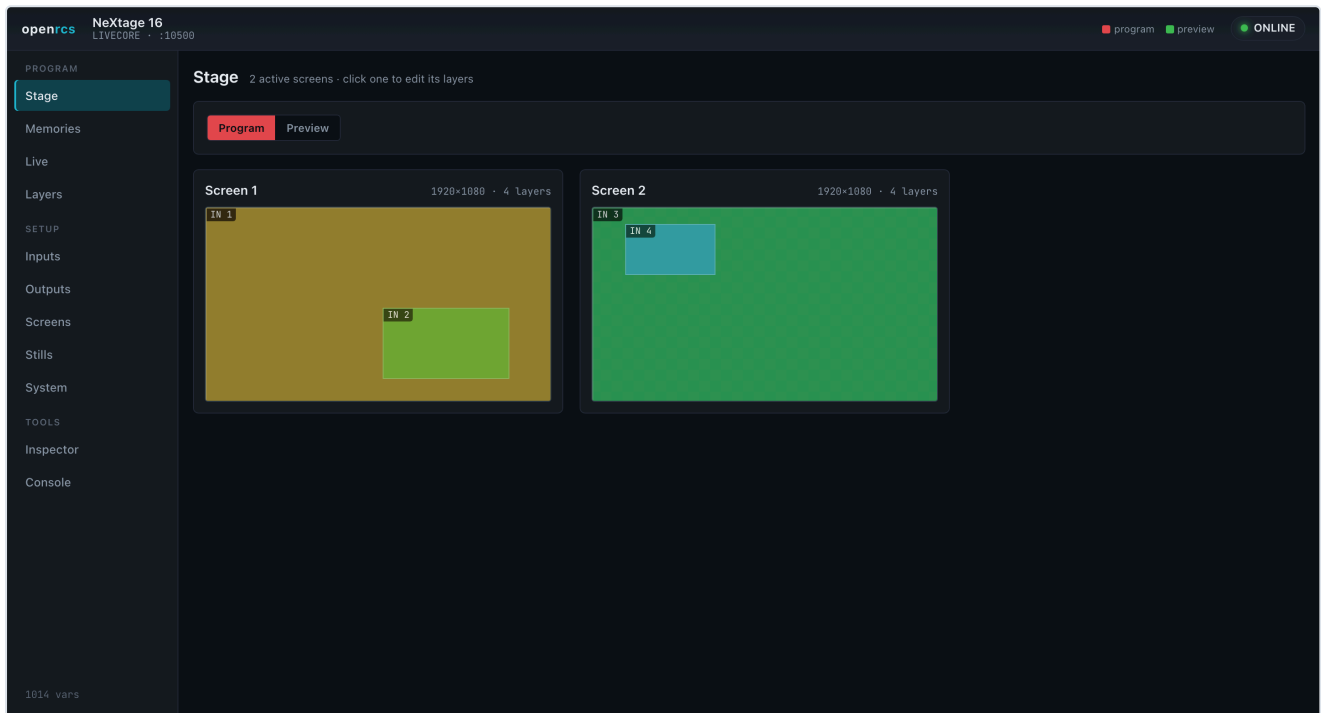
- **Drag a source onto a layer.** Drag from the palette onto a layer rectangle, onto a layer slot, or onto bare canvas — dropping on the canvas puts the source on the first free layer, where you let go. Clicking a source **arms** it instead, for touch panels: arm, then tap a layer. Arm — **none** — to clear a layer. The palette has **Inputs**, **Stills** and **Other** tabs on LiveCore.
- **Live thumbnails.** Where the device offers them (LiveCore inputs), the palette and the layers on the canvas show the actual picture on that input, refreshed a few times a minute. openrcs turns the device's snapshot system on for you.
- **Drag and resize** layers directly on each canvas, with every screen live at once, and use the **layout presets** — Fill, 2-up, 3-up, Quad, PiP, Stack — to arrange a screen's assigned sources into a common look in one click.
- **The layer properties panel** opens when you select a layer, and exposes everything the device holds for it: source and z-order, position and size (with keep-aspect, screen size, content size, a nine-point placement pad and aspect ratio presets), transparency and the master fader, cropping and aspect override, border style, colour, size and opacity, opening and closing transitions with their directions, timing and speed, the flying curve, and the effects — force transition, smooth move, flip, black & white, negative, sepia, solarise, strobe. A Midra unit shows its own equivalents instead, including per-layer opening and closing durations and a layer freeze.
- **Program sits above preview** for each screen, tagged red and green, with the live one marked **ON AIR**. Each canvas edits its own context, so you can build the next look underneath what is on air. **Show** picks which of the two you want; **Screens** toggles hide the ones you are not touching.
- **Take and Cut per screen**, plus a **T-bar** to run the transition by hand, a **step back** to the look before the last take, and — on a Midra — screen freeze and reload-program. **Take all** and **Cut all** in the top bar cover every visible screen, with a take time you set once.

On a LiveCore, which preset bank is program moves as you take; openrcs reads that from the device rather than assuming, so the ON AIR tag always follows the real output. If a transition stalls, a ... button appears to complete it.

- **Unusable sources are flagged.** A layer pointed at an input the frame does not have can never open, and a take waiting on it never lands — openrcs marks those layers amber and offers to clear them, rather than letting you discover it mid-show. On a Midra the same marking covers inputs with no signal, which the device silently refuses to put on a layer at all; if a drop does not take, the page says so instead of pretending it worked.

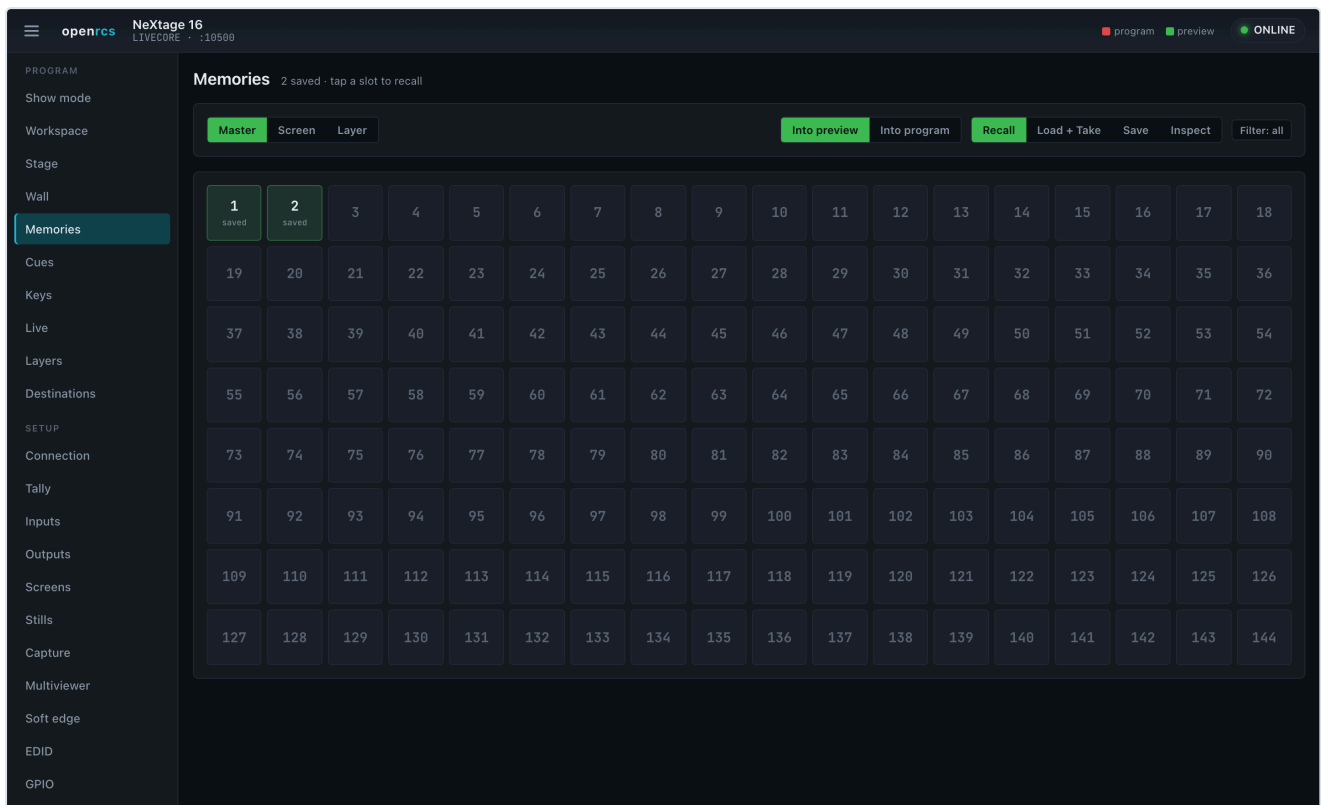
- **Memories** along the bottom. On LiveCore that is the device's 144 **screen memories** and 144 **master memories**, with save mode, load, load-and-take, erase, editable labels, and the per-category **filter** so a recall can bring in only the layer geometry, or only the sources, and so on. On a Midra it is the eight presets that live in the unit.

Stage



A mission-control overview: every active screen, drawn to scale with its live layers, in one place. Layers are coloured by source so the same input reads the same across screens. Toggle **Program / Preview**, and click any screen to jump straight into its layer editor.

Memories



Recall, store and take memories. Toggle between **Master** memories (whole-device presets across all screens) and **Screen** memories (per-screen presets), and choose what a slot tap does:

- **Recall** — load the memory into preview.
- **Load + Take** — load it and take it to program in one action.
- **Save** — store the current state into the slot you tap.

Saved slots light up, driven by the device's own validity flags, so the grid always reflects what is actually stored on the hardware. **Inspect** mode shows a scaled thumbnail of a stored memory's layout — source, size and position of every layer — without recalling it. On **Midra**, memories follow that family's model instead: eight preset slots, each captured from the live program, with the same inspect-thumbnail, recall and erase.

Layer memories

The third tab in Memories. Pick the layer to **capture from** — screen, program or preview, and which layer — then switch to **Capture** and tap a slot. The slot stores every property that layer has: 40 of them on a LiveCore, 27 on a Midra, read from the variable table your processor actually advertised rather than a fixed list.

To use one, set **Apply to** — any screen, either bank, any layer — and tap the slot. The category chips apply here exactly as they do to a screen memory, so you can drop a layer's position and size onto another layer without disturbing its source.

Two things worth knowing:

- **A recall is verified, not assumed.** openrcs re-reads the layer afterwards and names anything that did not land. A Midra silently refuses a source with no signal — no error, no echo — and this is the only way to see it.
- **A capture records which platform it came from** and refuses to apply on the other one. The two families spell the same properties with different ranges, so the numbers would land but mean something else.

The screenshot shows the openrcs NeXStage 16 LIVECORE interface. The sidebar on the left contains navigation options: PROGRAM, Show mode, Workspace, Stage, Wall, Memories (selected), Cues, Keys, Live, Layers, Destinations, SETUP, Connection, Tally, Inputs, Outputs, Screens, Stills, Capture, Multiviewer, Soft edge, EDID, and GPIO. The main area is titled 'Memories' and shows '3 of 50 layer memories saved - tap a slot to apply'. A button 'Applied memory 1 to Screen 1 layer 1' is visible. Below this, there are controls for 'Capture from' (Screen 1, Program, Preview, Layer 1) and 'Apply to' (Screen 1, Program, Preview, Layer 1). A note states: 'Neither platform stores a single layer, so this bank lives in the browser. It survives a reload, and a recall is checked by reading the layer back.' The 'Layer memory 1' section shows 'Wide PiP' and buttons for 'Apply to target' and 'Erase'. It also displays a message: '40 properties stored, 40 pass the current filter - captured 22/08/2026, 11:12:00 from Screen 1 layer 1 on a livecore'. A table lists 40 properties with columns for VARIABLE, PROPERTY, CATEGORY, and VALUE.

VARIABLE	PROPERTY	CATEGORY	VALUE
PRinp	PE_INPUTNUM	Source	25
PRalp	PE_ALPHA	Transparency	200
PRslh	PE_SIZEH	Pos/size	960
PRslv	PE_SIZEV	Pos/size	540
PRpoh	PE_POSH	Pos/size	32448
PRpov	PE_POSV	Pos/size	32588
PRbst	PE_BORDER_STYLE	Border	1
PRbcr	PE_BORDER_COLOR_RED	Border	255
PRbcg	PE_BORDER_COLOR_GREEN	Border	64
PRbcb	PE_BORDER_COLOR_BLUE	Border	0
PRbsh	PE_BORDER_SIZE_H	Border	12
PRbsv	PE_BORDER_SIZE_V	Border	12
PRbaL	PE_BORDER_ALPHA	Border	255

Video out — Midra

Most Midra frames carry a second output beside the numbered ones. On a frame where it is the SDI plug, this is the only SDI the unit has, and this view is where it lives.

Mode decides what the plug is for. It is not a display setting — it reassigns the plug:

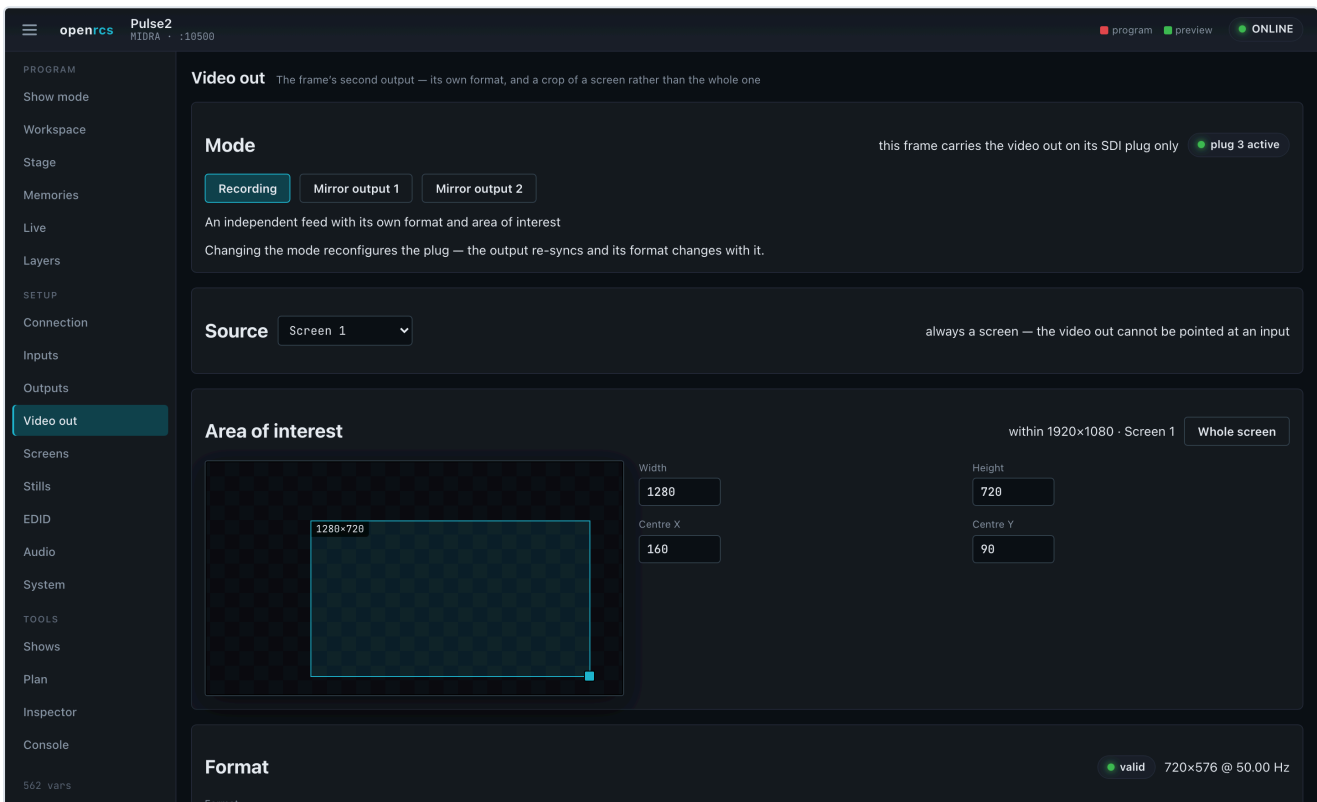
Mode	What the plug does
Recording	An independent feed with its own format and an area of interest. Standard definition only — 720×576 or 720×480, whatever the format list offers.
Mirror output 1	The plug becomes an extra plug of output 1, carrying that output's full raster. HD if the output is HD.
Mirror output 2	The same, for output 2.

Only the modes your frame offers are enabled; it reports which ones it has.

In **Recording** mode you also get a **source** — which screen the feed is a view of, including the two tiled combinations — and an **area of interest**: drag the rectangle, or type a size and centre, to send a crop of that screen instead of all of it.

The video out cannot be pointed at an input. Every source it offers is a screen. To put one input on that plug, put the input on a layer of a screen the video out is looking at.

There is no HD area of interest. The crop exists only in Recording mode, and Recording mode is standard definition. If you need a framed HD feed, mirror an output and constrain the composition with a **working area** instead.



Working area

Neither platform can crop an output at HD. When only part of a screen is actually seen, openrcs can constrain itself instead: set a **working area** on the screen in **Setup → Screens** and everything composes inside it.

- Layout presets divide the region rather than the raster — a quad is four cells of the part you see.
- A layer cannot be dragged, resized, typed or recalled outside it.
- Nothing is written to the processor. It never knows.

Shape presets cover the usual cases (16:9 or 4:3 centred, half the screen, centre 80%), or type the four numbers. Setting a region deliberately does not move anything that is already placed — **Fit**

existing layers does that when you want it, across every preset bank so a take cannot bring an overhang back.

openrcs

NeXtage 16

LIVECORE · :10500

program preview ONLINE

PROGRAM

Show mode

Workspace

Stage

Wall

Memories

Cues

Keys

Live

Layers

Destinations

SETUP

Connection

Tally

Inputs

Outputs

Screens

Stills

Capture

Multiviewer

Soft edge

EDID

GPIO

Screens

Output screens and their layer capacity - 1 with a working area

SCREEN	OUTPUT	SIZE (MODE)	MAX LAYERS	WORKING AREA	STATE
Screen 1	0	1×1	4	1536×864 @ 192,108	active
Screen 2	1	1×1	4	whole screen	active
Screen 3	2	1×1	0	whole screen	unused
Screen 4	3	1×1	0	whole screen	unused
Screen 5	4	1×1	0	whole screen	unused
Screen 6	5	1×1	0	whole screen	unused
Screen 7	6	1×1	0	whole screen	unused
Screen 8	7	1×1	0	whole screen	unused

Working area — screen 1

Screen 1 ▾

screen is 1920×1080

Fit existing layers

Use whole screen

Neither platform can crop an output, so this is openrcs's own constraint rather than a device setting: layers are kept inside the region and the processor is never told. Use it when only part of the screen is actually seen — an LED wall inside a larger canvas, or an SDI feed that has to stay within a frame.

1536×864

16:9 centred

4:3 centred

Left half

Right half

Centre 80%

X

Y

Width

Height

192

108

1536

864

Layouts divide this region, and a layer cannot be dragged, sized or recalled outside it. Layers placed before the region was drawn are left where they are — "Fit existing layers" pulls them in.

Cues

openrcs

NeXtage 16

LIVECORE · :10500

program preview ONLINE

PROGRAM

Stage

Memories

Cues

Keys

Live

Layers

SETUP

Inputs

Outputs

Screens

Stills

System

TOOLS

Inspector

Console

Cues

A show script — each cue recalls a memory and takes it

No cues yet

GO NEXT

ADD CUE

Master

Screen

Slot

Label

2

optional

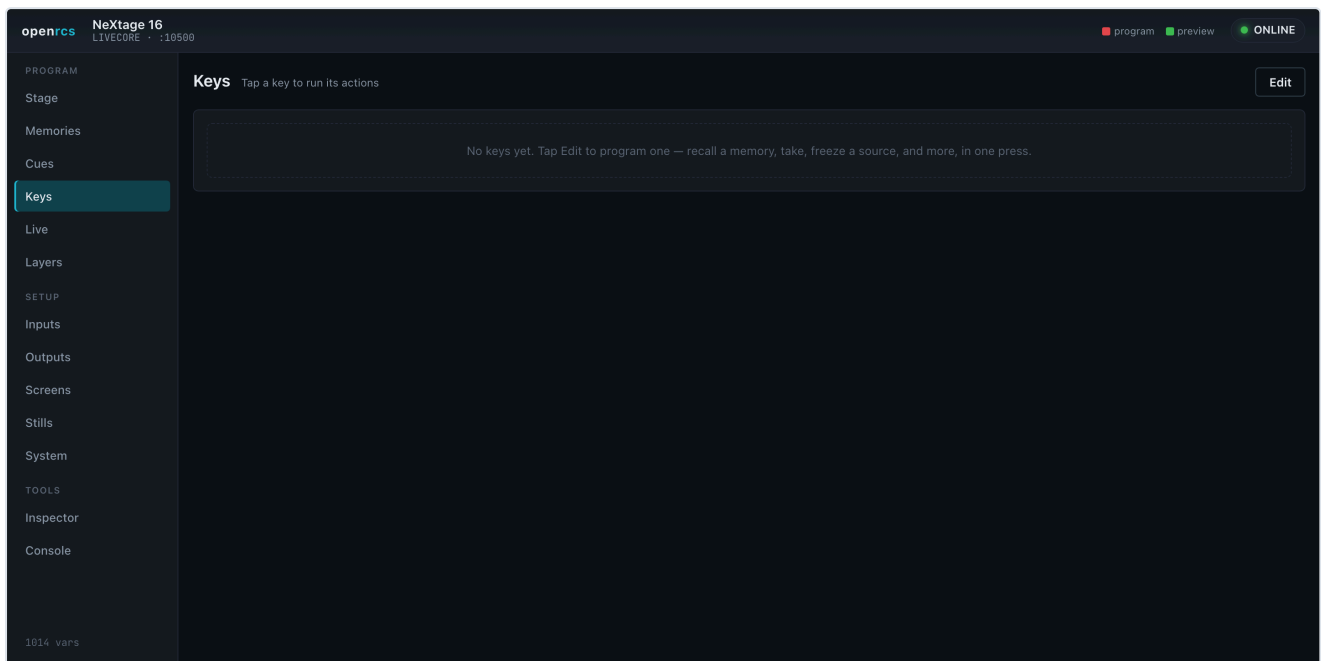
Add

CUE LIST (0)

Build a cue list from your saved memories, then run the show with GO NEXT.

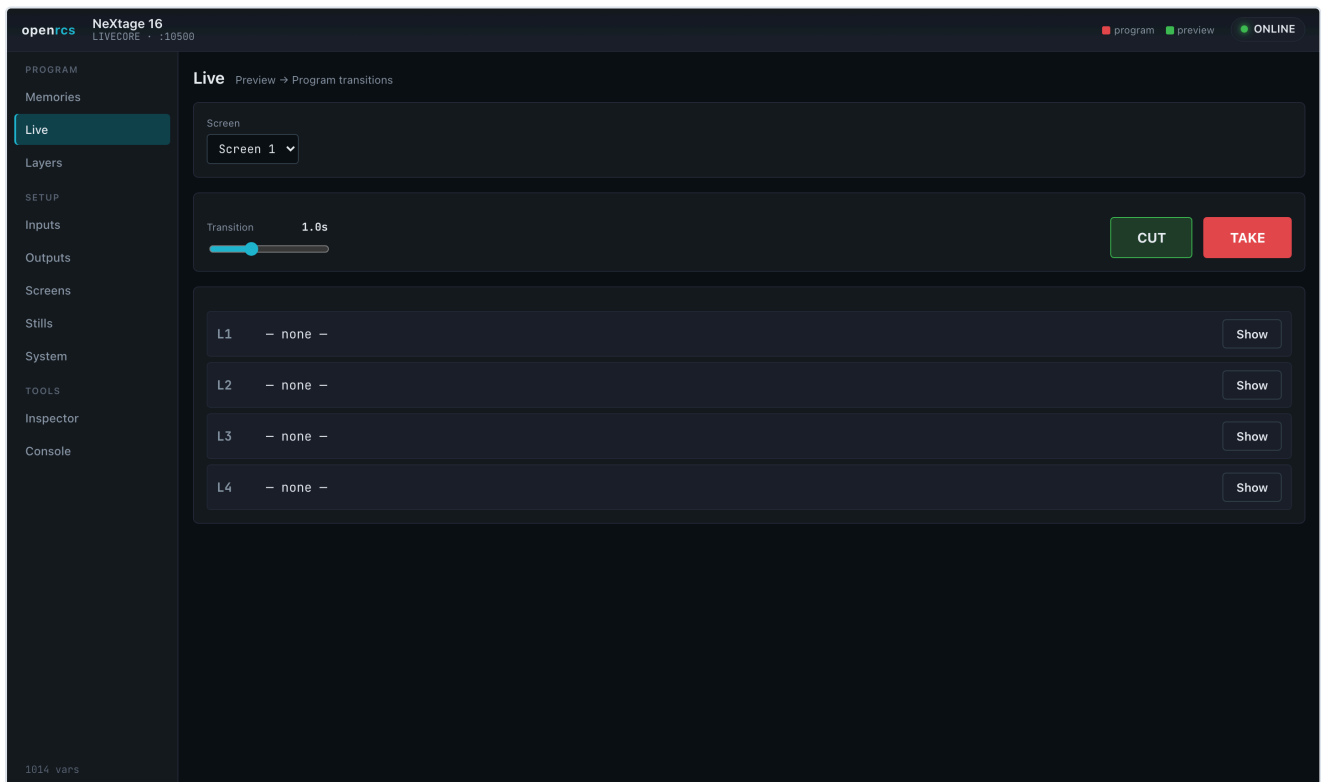
Turn your memories into a show script. Each cue recalls a master or screen memory and takes it; **GO NEXT** runs the list step by step. Preview or Go any cue directly, reorder and rename, and the current cue is highlighted. Cues are saved in the browser, so your running order survives a reload.

Keys



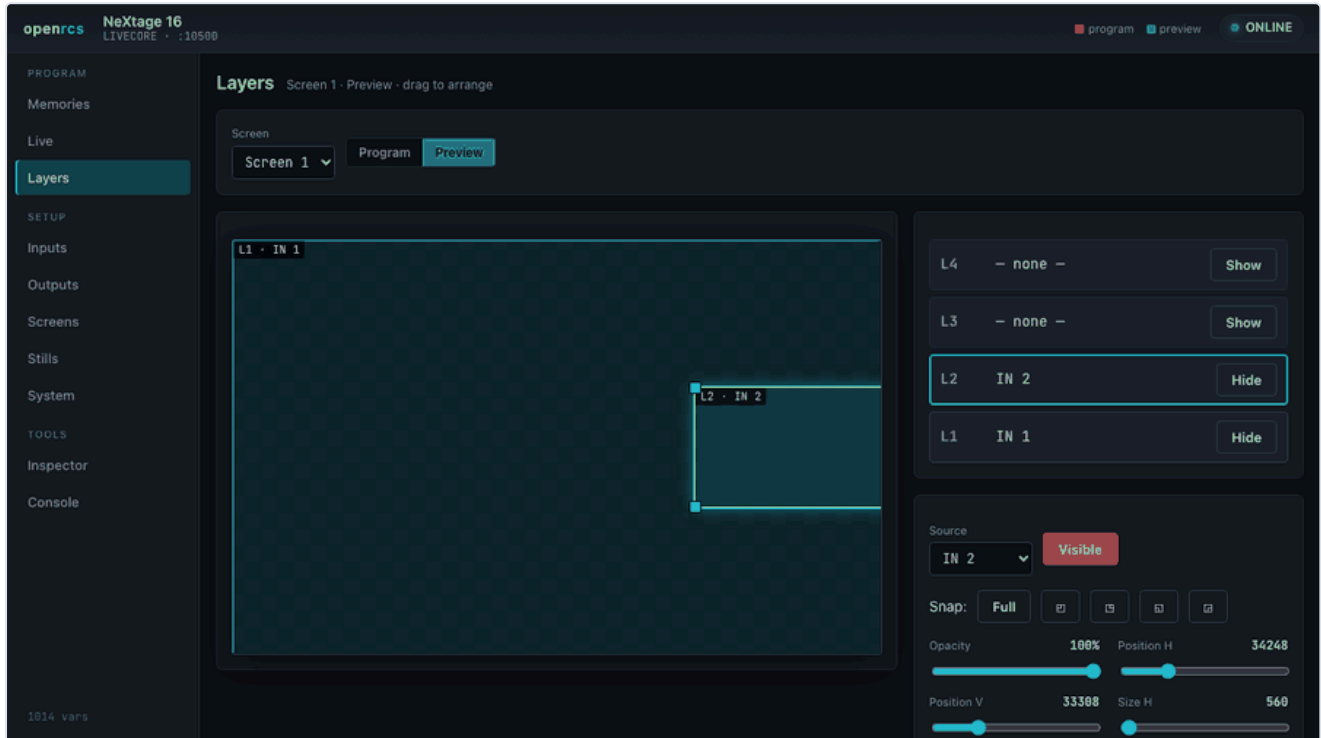
Programmable one-tap buttons. A key runs a sequence of actions — recall a memory (with take), take a screen or all screens, freeze an input, black an output, master-fade — so a whole cue-to-air, a panic black, or a freeze is a single press. Tap **Edit** to build them; they persist in the browser.

Live



The take bar: choose a screen, set a transition time, and move preview to program with **TAKE**, or switch instantly with **CUT**.

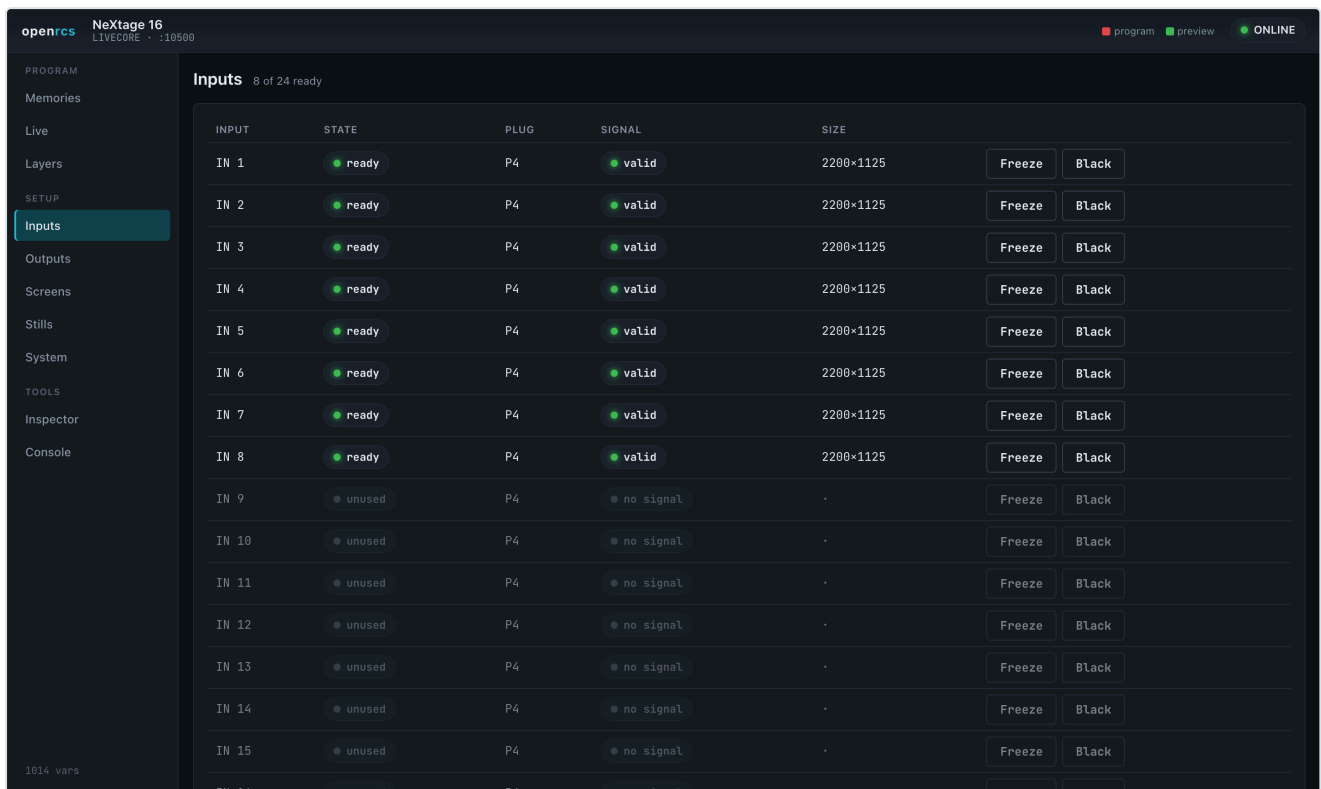
Layers



Arrange sources on the screen visually. Each layer is a rectangle on a canvas that represents the output: **drag to move**, **drag the corners to resize**, and use the snap presets to fill the screen or drop a layer into a quadrant. The layer stack on the right mirrors the canvas — assign a source and fine-tune position, size, opacity, border, crop and per-layer transitions, and raise or lower a layer in the stack. A **Program / Preview** toggle chooses which buffer you are editing, and each screen has a native **Background** (colour or a background set).

On **Midra**, a **Layout** picker offers the processor's built-in arrangements — choose one and the device lays the layers out for you, ready for sources.

Setup

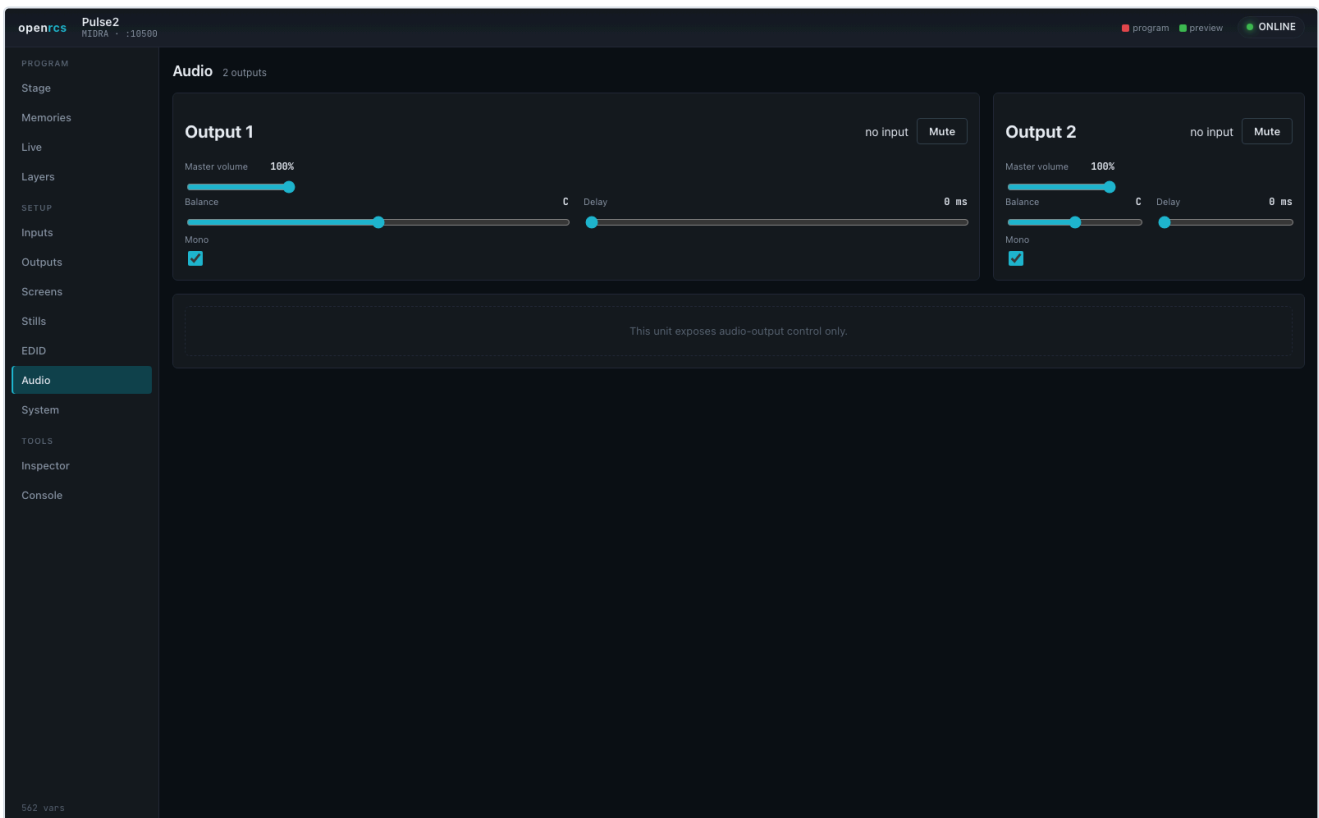
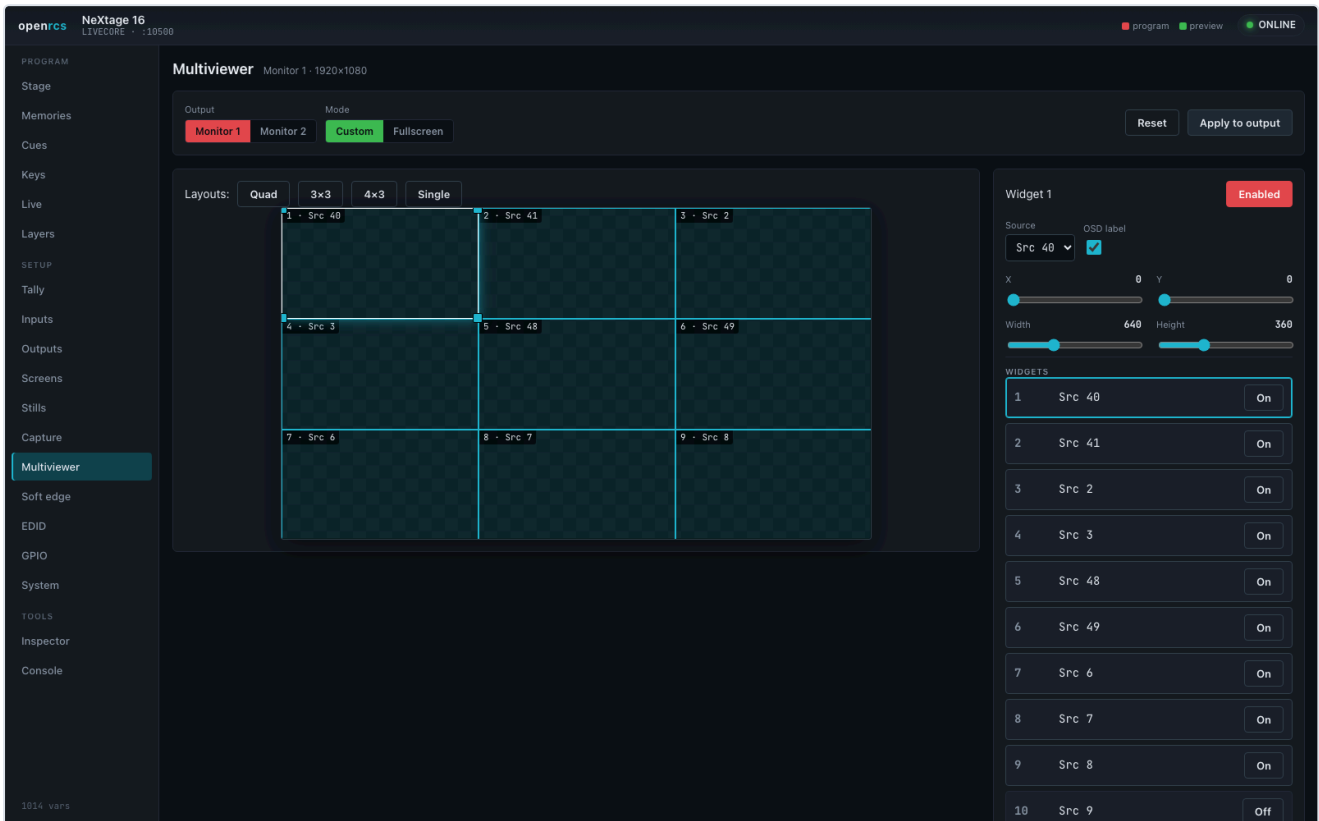


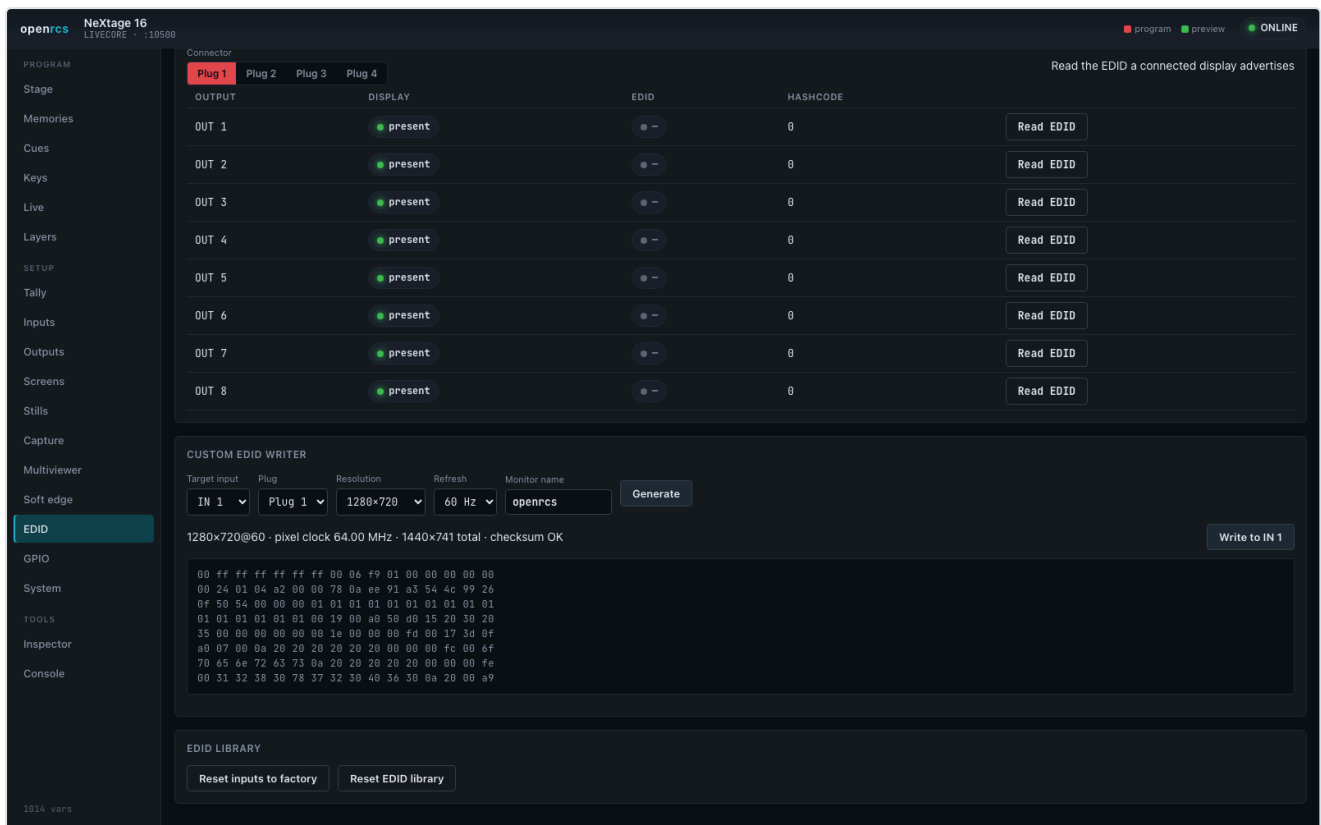
The Setup views cover configuration and monitoring. Which ones appear depends on the device:

- **Tally** — a live on-air grid: each source lights red on program, green on preview, straight from the device's own tally bus.
- **Inputs** — every input with its availability, active plug, live signal status and detected size, plus freeze and black.
- **Outputs** — the physical outputs, their connected displays, format, size, HDCP and output processing (brightness, contrast, gamma, gain).
- **Screens** — the output screens and their layer capacity.
- **Stills** — the still/logo library as a grid (LiveCore), or the frame store (Midra), showing what's stored and its size.
- **Capture** — grab a frame from a live source into the still library: pick a source and capture the full frame or a graphical region.
- **Multiviewer** — a drag-and-resize layout designer for the monitoring outputs: place up to twelve widgets, pick each one's source, and store layout memories.
- **Soft edge** — a per-edge blend editor for multi-output screens: click an edge to feather it into its neighbour and set the black level.
- **EDID** — set an input's preferred format and read the EDID a connected display advertises. The **custom-EDID writer** builds a valid EDID for any resolution and refresh rate and writes it to an input, so a source outputs exactly what you want.
- **Audio** — output volume, balance, delay and mute, plus per-input channel levels where the device provides them.

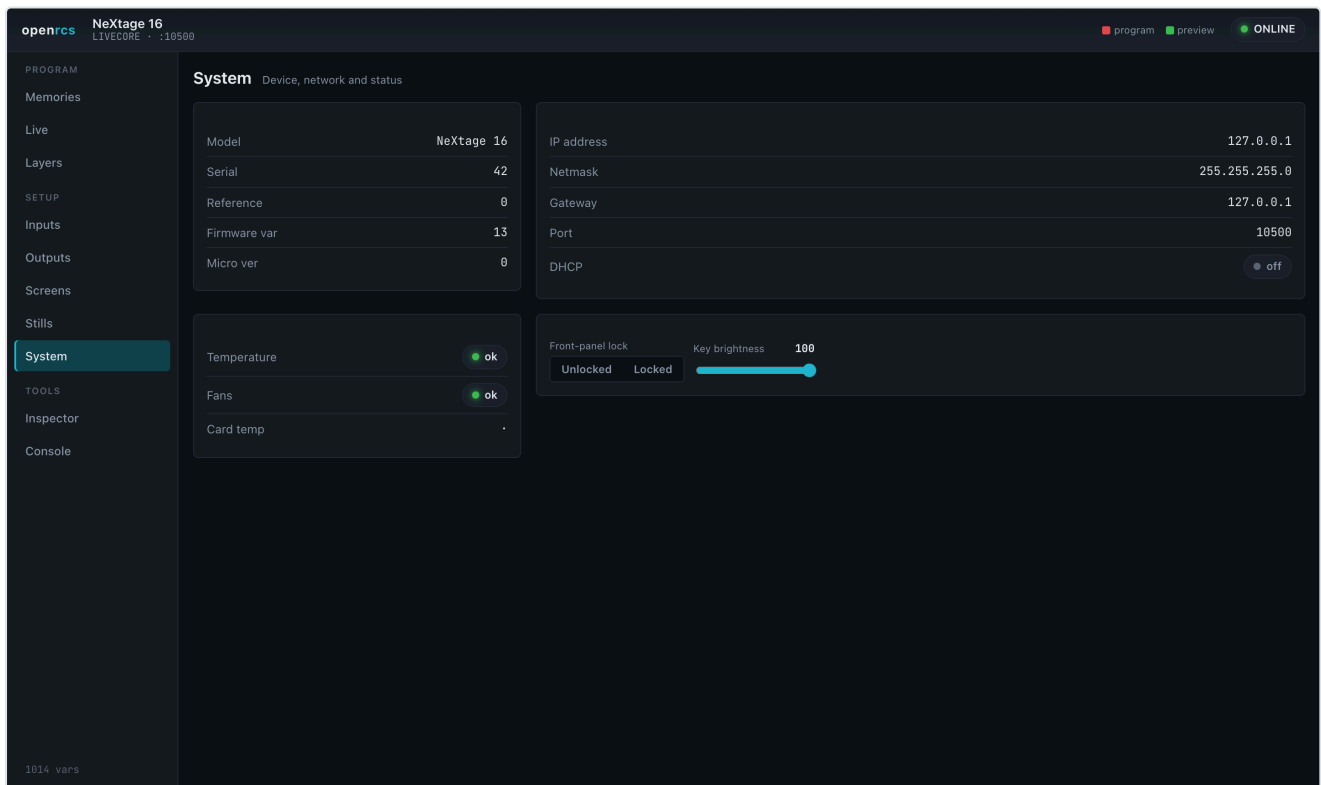
- **GPIO** — trigger inputs and tally/relay outputs.

Inputs and outputs also carry image adjustment: click an input row for brightness, contrast, colour, hue, RGB gain and crop; each output has its own processing and a format/rate selector.





System



Device identity, network settings, temperature and fan health, and front-panel lock and brightness.

Tools — Inspector and Console

- **Inspector** — search, read and set *any* of the device's variables. Useful for anything the dedicated views don't yet cover.
- **Console** — the raw protocol, sent and received, for diagnostics.

Sharing a view

Each view has its own URL (`.../#layers` , `.../#memories` , and so on), so you can bookmark or link straight to the panel you want.

Notes

Both families have been driven against real hardware — a NeXtage 16 (LiveCore) and a Pulse2 (Midra). A few behaviours still depend on the device: assigning a live input needs a signal present on it, and some capabilities vary by model and firmware (openrcs hides what a given unit doesn't implement). Per-variable ranges are the device's declarations — the hardware is always the final authority. The full protocol is documented in the [openrcs-protocol](#) reference.

openRCS · v0.4.0 · guide updated 2026-08-21 · stoatworks-labs.com/software/openrcs/

Latest version of this guide: stoatworks-labs.com/software/openrcs/guide/ · Source and issues:

<https://github.com/stoatworks-labs/openrcs>